



Tecnológico de Monterrey

E2. Proyecto Integrador

Programación orientada a objetos (Gpo 306)

TC1030.306

Federico Manuel Fernández Peña

A01178985

5 de Junio, 2026

Professora: Gabriela Zamora Leal

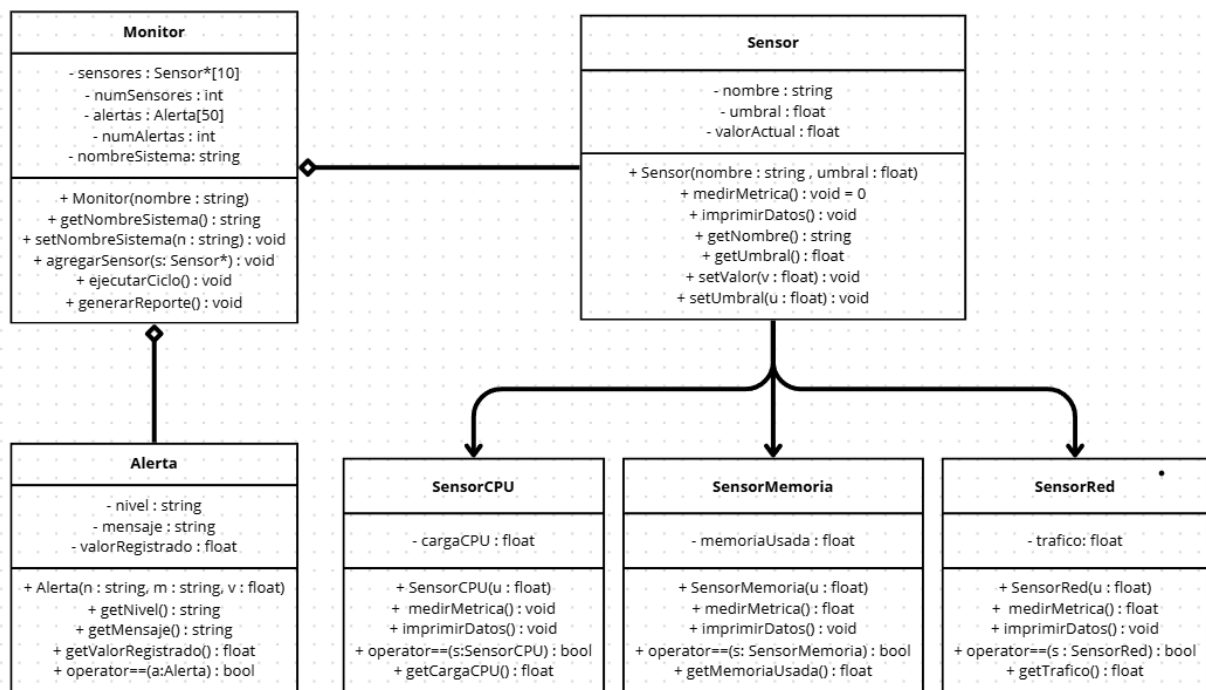
1. Introducción

Para propósitos de esta evidencia, decidí hacer un proyecto relativamente complicado. Fue un proyecto que demuestra mis conocimientos que he conseguido desde el semestre pasado con mi clase de fundamentos de programación en Python, y los conocimientos que he aprendido de C++ y Programación Orientada a Objetos. Quiero que este proyecto demuestre las habilidades que tengo como programador y poder presentarlo como un proyecto de curriculum. Este proyecto encapsula todo lo aprendido en esta materia, y trata de demostrar el gran potencial que puede tener, esto debido a que la programación orientada a objetos abarca muchos temas

Por eso decidí que el proyecto que realice, tuviera un uso real. En entornos tecnológicos, donde hay producción de software, centros de datos, etc. El monitoreo de recursos del sistema es indispensable para poder garantizar el máximo rendimiento posible. Por eso decidí hacer un sistema de monitoreo que denominé como “*Data Stream Monitor*”. Este programa monitorea los estados de los ordenadores en una compañía. Por limitaciones del proyecto, hice una simulación de lo que podría ser, usando datos aleatoriamente generados. Si decido desarrollar más a fondo el proyecto podría hacer que lea los datos reales de las computadoras registradas.

Explicando más a detalle mi proyecto, lo que hace la aplicación es que en base a límites establecidos por los administradores limitan el estado del CPU, RAM, y tráfico de red de los ordenadores. En caso de que el umbral establecido se haya excedido. Se va a emitir una alarma a los administradores para poder arreglar el problema. Si llegara a desarrollar más el programa, podría detectar los datos de los ordenadores en tiempo real y hacerlo genuinamente útil en un ámbito real.

2. Diagrama de Clases UML y Argumentación del Diseño



Decisiones de diseño clave:

- Clase abstracta Sensor como raíz de la jerarquía.

Elegí, una clase abstracta en lugar de una de interfaz para poder reutilizar los atributos comunes (nombre, umbral, valorActual) y métodos concretos (imprimirDatos, setters/getters), de esta manera evitando duplicar un código por cada subclase

- Función virtual pura medirMetrica()

Esta va obligar a que cada sensor concreto vaya a proveer su propia lógica de medición, esto va garantizar que no se vaya a instanciar un sensor genérico sin su implementación.

- Monitor como composición.

Monitor contiene arreglos de Sensor y Alerta, no hereda de ellos, esto va a respetar el principio de “componer antes que heredar” esto debido a que monitor no es un tipo de Sensor sino una clase que va a gestionar las clases de Sensor

- Alerta como clase de valor.

Como no necesita herencia y polimorfismo, la alerta se diseña como una clase concreta sencilla con tres atributos y con la sobrecarga de operador== para facilitar las comparaciones

- Uso de Arreglos estáticos

Este proyecto funcionará mejor si tuviera vectores, sin embargo para facilitar este proyecto decidí usar arreglos fijos. (Sensor*[10] y Alerta[50]).

3. Ejemplo de Ejecución

```
C:\Users\fedef\CLionProjects\P00Sem2\Evidencia2\cmake-build-debug\Evidencia2.exe
```

```
=====
```

```
DATA STREAM MONITOR
```

```
=====
```

```
>>> CASO 1: Creando objetos derivados y mostrando datos
```

```
Datos iniciales de los sensores:
```

```
Sensor CPU -> Carga Actual: 37% | Umbral Configurado: 75%
```

```
Sensor Memoria -> RAM Usada: 32% | Umbral Configurado: 80%
```

```
Sensor Red -> Trafico Actual: 21 MB/s | Umbral Configurado: 50 MB/s
```

```
-----
```

```
>>> CASO 2: Calculando metricas con metodos sobrescritos (.medirMetrica())
```

```
Metrica calculada individualmente para CPU: 50%
```

```
Metrica calculada individualmente para RAM: 60%
```

```
-----
```

```
>>> CASO 3: Demostrando polimorfismo con el Monitor
```

```
=== Ejecutando ciclo de monitoreo para: Server-DataCenter-01 ===
```

```
Sensor [CPU] midio: 64
```

```
Sensor [Memoria] midio: 95
```

```
-> ALERTA GENERADA en Memoria!
```

```
Sensor [Red] midio: 43
```

```
=====
```

```
=== Ejecutando ciclo de monitoreo para: Server-DataCenter-01 ===
```

```
Sensor [CPU] midio: 86
```

```
-> ALERTA GENERADA en CPU!
```

```
Sensor [Memoria] midio: 45
```

```
Sensor [Red] midio: 36
```

```
=====
```

```
===== REPORTE DE ALERTAS - SISTEMA: Server-DataCenter-01 =====
```

```
Total de alertas registradas: 2
```

```
Alerta 1 -> Nivel: CRITICO | Mensaje: El valor supero el umbral tolerado |  
Valor: 95
```

```
Alerta 2 -> Nivel: CRITICO | Mensaje: El valor supero el umbral tolerado |  
Valor: 86
```

```
=====
```

```
-----
```

```
>>> CASO 4: Usando la sobrecarga de operadores (==)
```

```
Comparando si cpuA tiene la misma carga actual que cpuB...
```

```
Resultado: Los sensores de CPU tienen cargas distintas actuales.
```

```
=====
```

```
Process finished with exit code 0
```

La salida del código main que yo hice cumple con los casos requeridos por las instrucciones siendo

1. Que los objetos derivados se instancian correctamente
2. mi método medirMetrica() invoca la versión sobreescrita de cada clase
3. el Monitor usa polimorfismo en los sensores y registra alertas de cuando el valor ya supera el umbral establecido
4. Operator== compara correctamente el estado de dos objetos de Sensor CPU

4. Argumentación

4.1 Clases utilizadas

Clase	Funcion
Sensor (abstracta)	Modelo base para cualquier derivada de Sensor, define la interfaz comun
SensorCPU	Simula y reporta el porcentaje de uso en el CPU
SensorMemoria	Simula y reporta el porcentaje de uso de RAM
SensorRed	Simula y reporta el porcentaje de uso en el la red en MB/s
Alerta	Encapsula un evento de umbral superado (nivel, mensaje, valor)
Monitor	Administra los sensores, y genera reportes de alertas usando bucles

4.2 Empleo del concepto de la herencia

Las tres clases de sensor se heredan públicamente de Sensor. Cada constructor derivado invoca al constructor mediante lista de inicialización ejemplo: Sensor(“CPU”, u). Esto garantiza que los atributos heredados se inicializan correctamente antes de ejecutar el constructor derivado.

4.3 Empleo del concepto de los modificadores de acceso

Miembro	Modificador	Explicación
nombre, umbral, valorActual	private	Solo accesibles con setters/getters, para evitar modificación accidental
cargaCPU, memoriaUsada, tráfico	private	Estado interno de cada sensor, no debe ser visible
medirMetrica(), imprimirDatos()	public	Forman la interfaz que Monitor necesita
nivel, mensaje, valorRegistrado	private	Son inmutables por la construcción, solo se puede leer con getters

4.4 Empleo de la Sobrecarga y Sobreescritura de Métodos

Cada clase derivada sobreescrive `medirMetrica()` e `imprimirDatos()`. La palabra `override` se declara en los `.h` de las clases derivadas, esto permite que el compilador verifique que realmente se está sobreescribiendo y no creando un nuevo método por error. Opte por sobreescibir también `imprimirDatos()` en cada subclase para mostrar unidades y etiquetas específicas (% , MB/s), aunque el método base ya proporciona un versión genérica funcional. La clase `Alerta` posee dos constructores: el básico (sin parámetros) y el parametrizado `Alerta(string n, string m, float v)`. Esto va permitir crear objetos vacíos como objetos parametrizados

4.5 Empleo del concepto del Polimorfismo

El poliformismo se aplica en el método `ejecutarCiclo()` de `monitor`, el arreglo de sensores que establece almacenar punteros a la clase base. En tiempo de ejecución, la `vtable` de C++ resuelve la llamada de `medirMetrica()` hacia la implementación correcta (`SensorCPU`, `SensorMemoria` o `SensorRed`) sin que la clase `Monitor` necesite conocer el tipo concreto. Esto va permitir agregar nuevos tipos de sensores en el futuro sin tener que modificar la clase `Monitor`.

4.6 Empleo del concepto de las Clases Abstractas

La clase `sensor` es abstracta debido a la función virtual pura que estableció en `medirMetrica()`, esto hace que sea imposible instanciar directamente un `Sensor` genérico (el compilador generará un error), lo que tiene sentido considerando que un sensor sin tipo no sabe que medir. También obligaría a cualquier clase derivar a implementar `medirMetrica()` de esta manera garantizando que el contrato de la interfaz siempre se cumpla

4.7 Empleo de las sobrecargas

Clase	Criterio de igualdad
SensorCPU	Mismo valor de cargaCPU
SensorMemoria	Mismo valor de memoriaUsada
Alerta	Mismo nivel + mensaje + valorRegistrado

5. Casos que harían fallar el proyecto

1. Exceder el arreglo de Sensores
 - a. Como fueron establecidos 10 por temas de limitaciones, si el contador de cantidadSensores llega a 10 o más va mostrar un error
2. Exceder el arreglo de alertas
 - a. Como fueron establecidos 50 por temas de limitaciones, si se generan mas de 50 alertas el contador se detiene y las nuevas alertas se terminan perdiendo

6. Conclusión

Este proyecto para mi fue ambicioso, considerando el tiempo que tenía para hacer el proyecto, y considerando las entregas que se me estaban acumulando. Decidir hacer un proyecto pesado en poco tiempo fue una decisión extrema pero me gustó. Este proyecto me sirvió para poder demostrar mis habilidades que tengo en la programación en general. Con más tiempo me gustaría desarrollarlo mucho más, siento que esto es algo que puede tener una aplicación real en alguna compañía pequeña si lo ocupa, ya que compañías internacionales seguramente ya tienen programas que hagan esto. Pero aplicar un programa así en una escala local puede ser mejor. Y es lo que me gustaría mejorar, hacerlo más realista, no tener que usar valores aleatoriamente generados sino usar datos reales de ordenadores reales. Esto también me traería complicaciones porque va ver cosas nuevas que tendré que aprender pero que estoy sumamente dispuesto a hacer.